Software Architecture 2024 [2026]

Lecture 2, part 2/2

Software quality and architectural significance

Wouter Joosen

Laurens Sion

Dimitri Van Landuyt

KU LEUVEN DistriNet

Assignment 1

A. **Utility tree** with **ASRs**

- Description/summary incl. metric or objective criterion
- Prioritization [business value, architectural impact] (H,M,L) and motivation why

B. **[Three] QASs**

- Highly ranked scenarios (preferably [and obviously])
- Different types (if possible)
- Detailed and in-depth

- **Team assignment** on Toledo
  - LaTeX template to help you structure the utility tree and the QASs
  - Initial examples provided
  - Relevant LaTeX commands (from saasrs.sty) \qualityattributescenario and \insertASR
- Deadline: **[19/03]**

ASRs??
QASs??

It’s all about

Quality attributes

Quality attributes

1. **Different “non-functional requirements”**

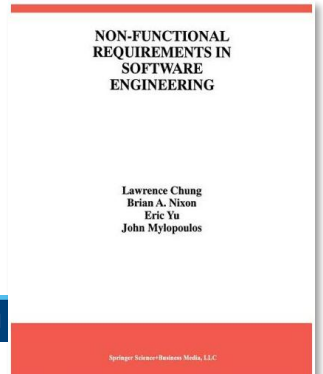
2. **Perceived as ‘desirable’ by stakeholders**: business value

3. **Impactful requirements**: architectural impact

Non-functional requirements (NFRs)

...address important issues of quality for software systems

- If NFRs are not addressed, the results can include:
  - software which is inconsistent and of poor quality;
  - users, clients and developers who are unsatisfied; and
  - time and cost overruns to fix software which was not developed with NFRs in mind.

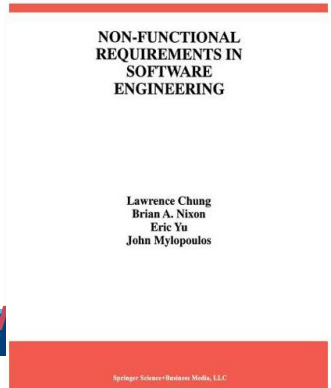


What about these “non-functional requirements”?

Non-functional requirements

- ... are **difficult to test**; therefore, they are usually evaluated **subjectively**
- ... can be subjective, since they can be viewed, interpreted and evaluated differently by different people. **Since NFRs are often stated briefly and vaguely, this problem is compounded**
- ... can also be **relative**, since the interpretation and importance of NFRs may vary depending on the particular system being considered
- ... can often be **interacting**: attempts to achieve one NFR can hurt or help the of other NFRs
- ... have a **global impact on systems**, and localized solutions may not suffice

For all these reasons, non-functional requirements can be **difficult to deal w**



Quality attributes

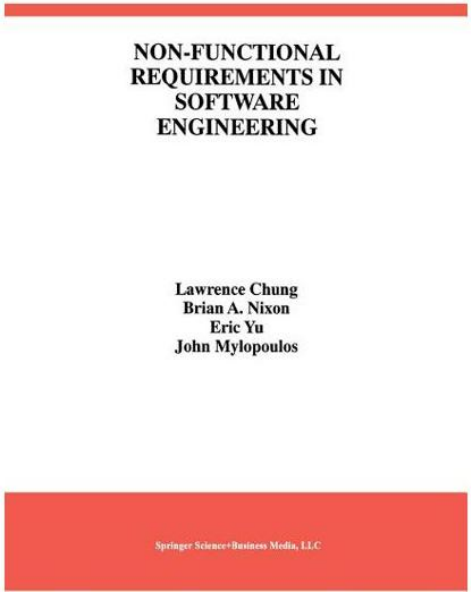
- Business qualities
  - (Engineering/operating) Cost, time to market, projected lifetime, integration with legacy systems
- *System qualities*
  - Performance, Availability, Scalability/elasticity, Mobility, Monitorability, Interoperability, Security, Usability, Monitorability
- *Design/constructive quality*
  - Variability, Testability, Deployability, Mobility, Development Distributability
- Internal quality
  - Conceptual integrity (impacts modifiability), correctness and completeness, build-ability (related to cost), modularity, …

accessibility,
adaptability,
affordability,
availability,
capacity,
cohesiveness,
communication time,
component integration cost,
comprehensibility,
confidentiality,
controllability,
correctness,
customer evaluation time,
data-space performance,
dependability,
distributivity,
domain analysis time,
enhanceability,
extensibility,
feasibility,
generality,
impact analyzability,
inspection cost,
inter-operability,
learnability,
maintenance cost,
mean performance,
modifiability,
nomadicity,
operability,
performability,
planning time,
precision,
productivity,
promptness,
reconfigurability,
reengineering cost,
replaceability,
responsiveness,
risk analysis cost,
safety,
security,
simplicity,
space boundedness,
stability,
supportability,
susceptibility,
testing time,
timeliness,
trainability,
understandability,
usability,
variability,
visibility,

accountability,
additivity,
agility,
buffer space performance,
clarity,
commonality,
compatibility,
component integration time,
conceptuality,
configurability,
coordination cost,
cost,
customer loyalty,
decomposability,
development cost,
diversity,
efficiency,
evolvability,
external consistency,
flexibility,
guidance,
independence,
inspection time,
internal consistency,
main-memory performance,
maintenance time,
measurability,
modularity,
observability,
operating cost,
performance,
plasticity,
predictability,
project stability,
prototyping cost,
recoverability,
reliability,
replicability,
retirement cost,
risk analysis time,
scalability,
sensitivity,
software cost,
space performance,
standardizability,
surety,
sustainability,
throughput,
tolerance,
transferability,
uniform performance,
user-friendliness,
verifiability,
wrappability.

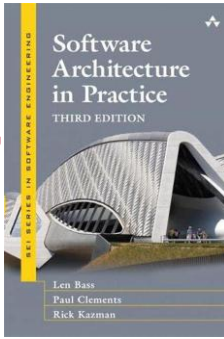
accuracy,
adjustability,
auditability,
capability,
code-space performance,
communication cost,
completeness,
composability,
conciseness,
consistency,
coordination time,
coupling,
customizability,
degradation of service,
development time,
domain analysis cost,
elasticity,
execution cost,
fault-tolerance,
formality,
hardware cost,
informativeness,
integrity,
intuitiveness,
maintainability,
maturity,
mobility,
naturalness,
off-peak-period performance,
peak-period performance,
planning cost,
portability,
process management time,
project tracking cost,
prototyping time,
recovery,
repeatability,
response time,
reusability,
robustness,
secondary-storage performance,
similarity,
software production time,
specificity,
subjectivity,
survivability,
testability,
time performance,
traceabi lity,
transparency,
uniformity,
validity,
versatility,

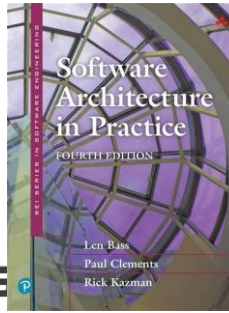
Table 5.2. A list of non-functional requirements.



Quality attributes

- **Handbook**:
  - Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Usability, Testability*
  - Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*
  - *4th edition*: *Safety, Energy Efficiency, Integrability, Buildability, Conceptual integrity, Marketability*





Part I.
How to identify relevant quality attribute requirements for your application?

Three complementary strategies

Identify relevant quality attributes at the basis of

1. …*the functionality of the system*
2. …*business goals*
3. …types of *quality attributes*

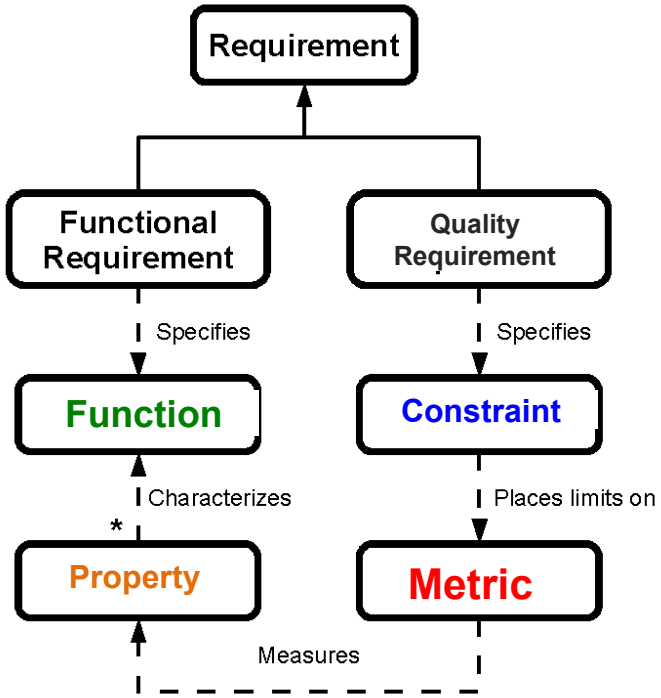
Strategy 1. Identify quality requirements at the basis of

.. the **functionality** of the system

Wait… didn’t we just call these “NON-functional” requirements?

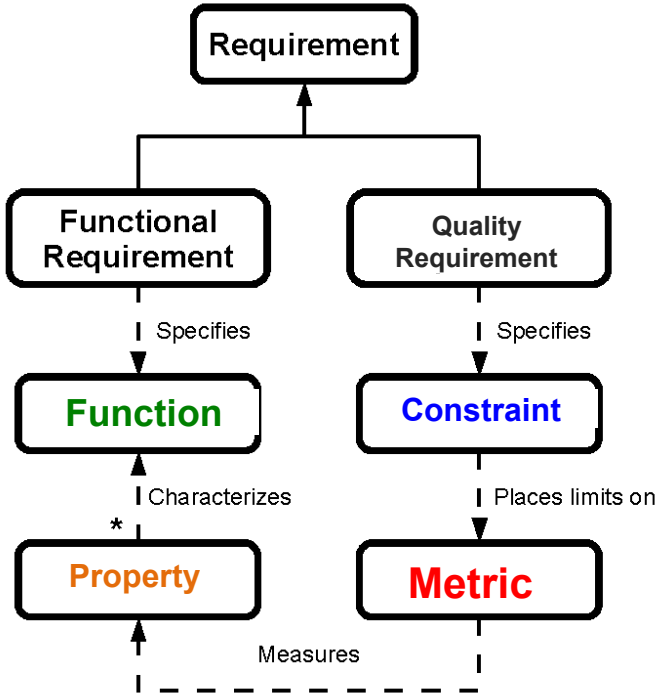
Quality <> Functionality

Relation between Functionality and Quality



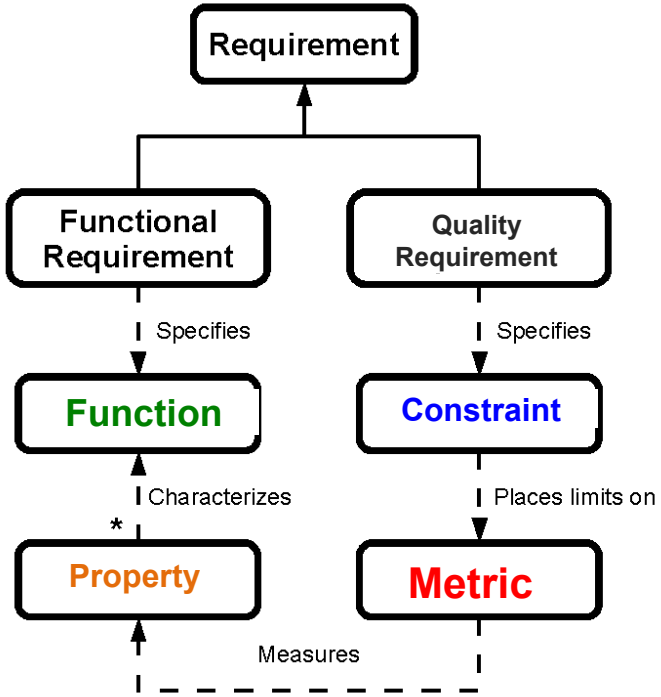
- *[functionality]* Through the portal, n customers can simultaneously browse the catalog of items on sale
- *[performance]* At least 10,000 customers should be able to access the portal at the same time

Relation between Functionality and Quality



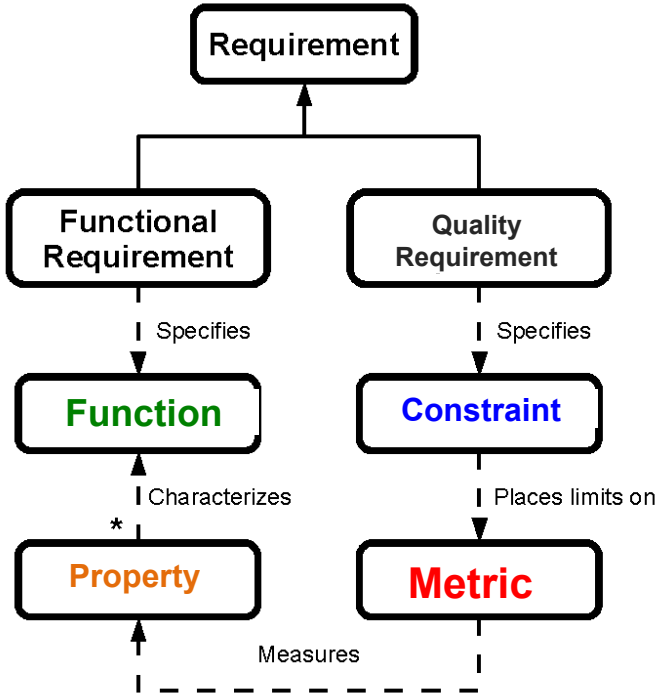
- *[functionality]* Through the portal, customers can browse the catalog to find an item that meets their criteria
- *[usability]* The task time *to* find a specific item should be no less than 3 minutes

Relation between Functionality and Quality



- *[functionality]* Through the portal, customers can browse the catalog of items on sale
- *[security]* *only registered and authenticated customers* should be able to browse

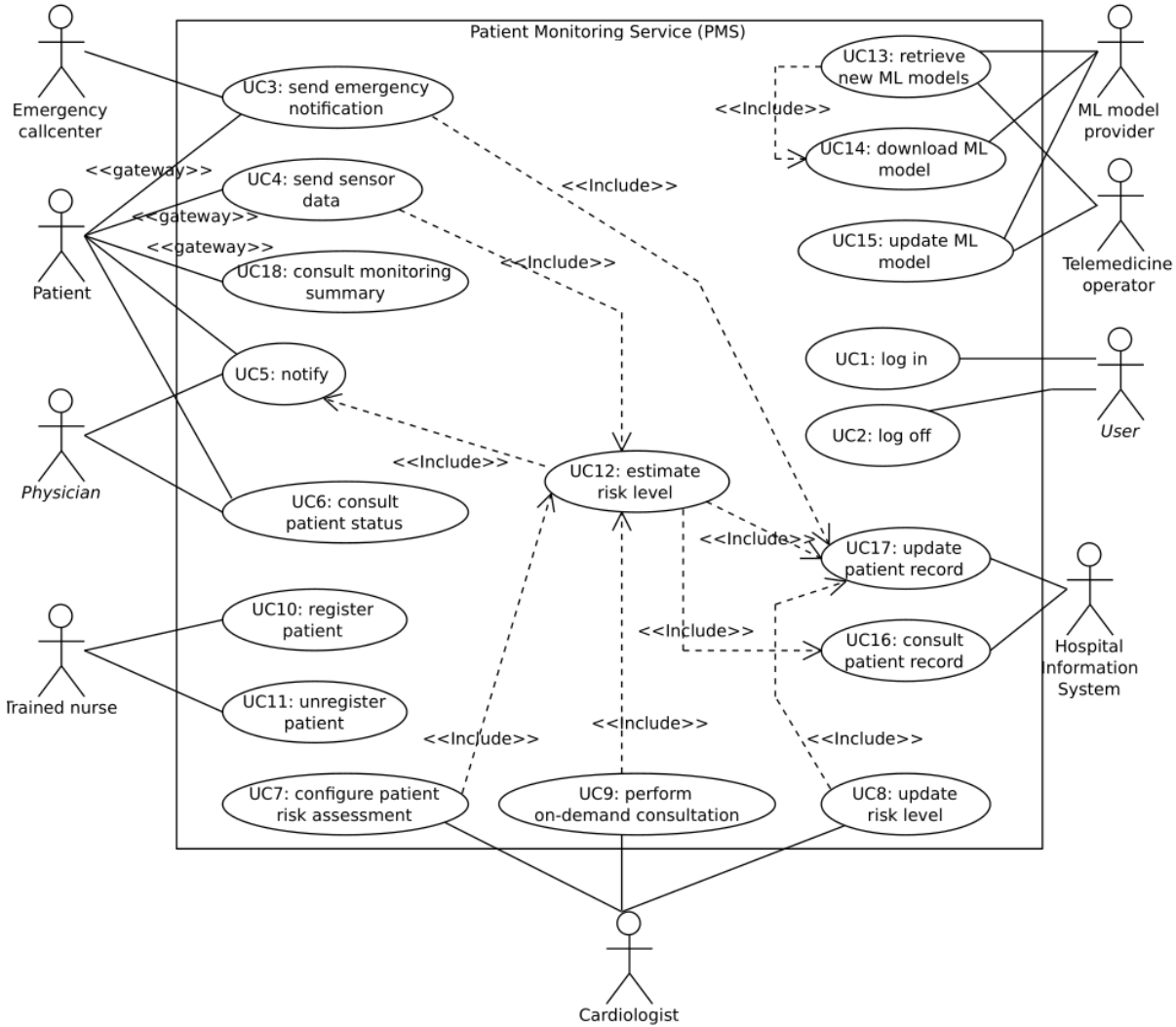
Relation between Functionality and Quality



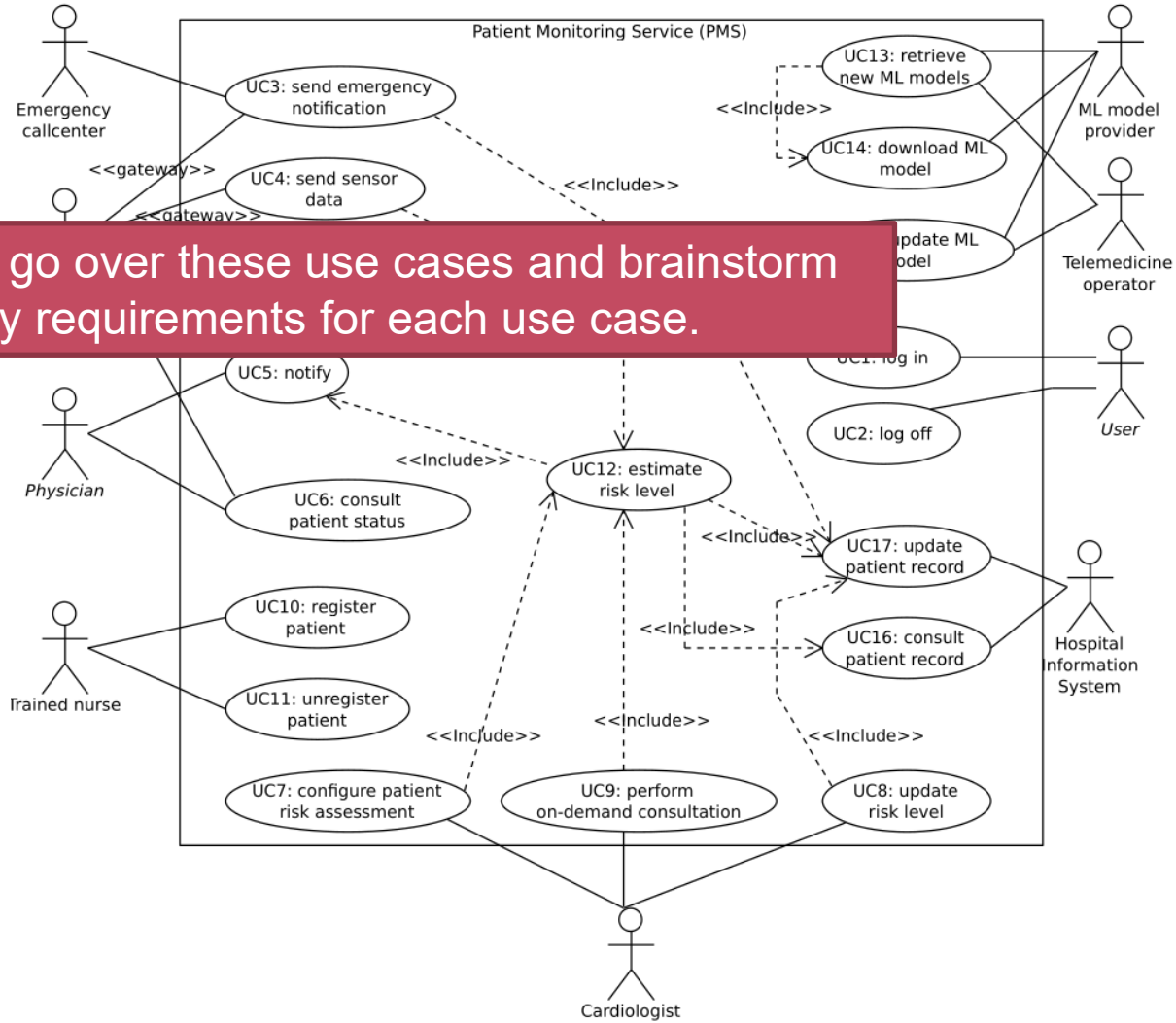
- *[functionality]* Through the portal, customers can browse the catalog of items on sale
- *[availability] the functionality to browse for item should be available for 99.99%*

This means that ..

you will need to understand the functional requirements very well, in order to get a grip on the non-functional requirements


Patient Monitoring Service (PMS)
Emergency callcenter
Patient
Physician
Trained nurse
Cardiologist
ML model provider
Telemedicine operator
User
Hospital Information System
UC3: send emergency notification
UC4: send sensor data
UC18: consult monitoring summary
UC5: notify
UC6: consult patient status
UC10: register patient
UC11: unregister patient
UC7: configure patient risk assessment
UC9: perform on-demand consultation
UC8: update risk level
UC12: estimate risk level
UC13: retrieve new ML models
UC14: download ML model
UC15: update ML model
UC1: log in
UC2: log off
UC17: update patient record
UC16: consult patient record
<<gateway>>
<<Include>>

Strategy: you can go over these use cases and brainstorm about quality requirements for each use case.



Preview / think about this

Quality requirements add relevant constraints to functional requirements

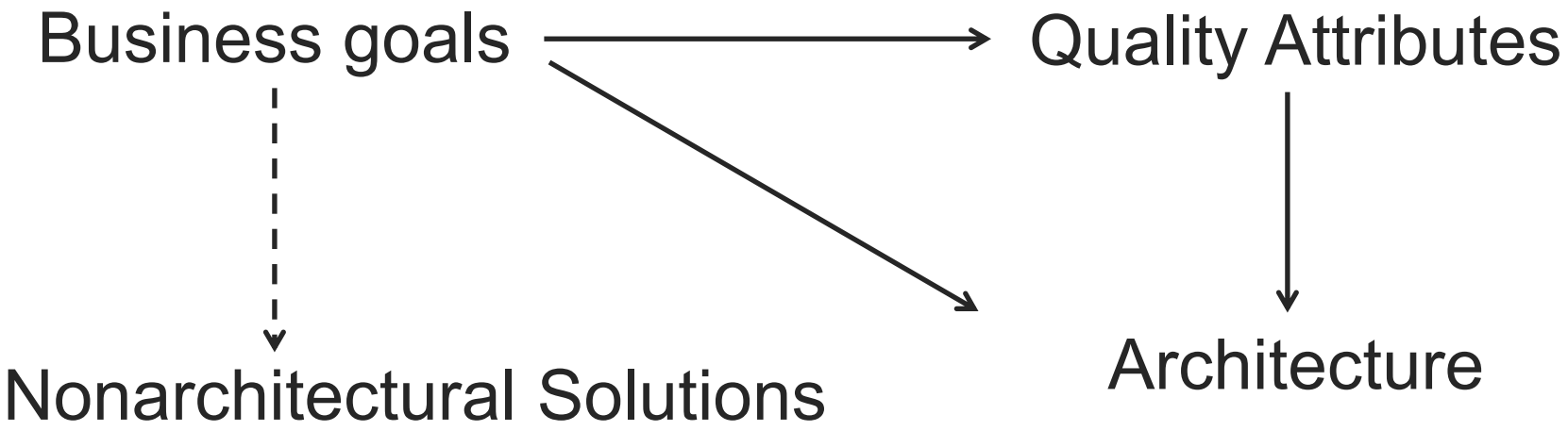
- … but they can **also** introduce new functional requirements
  - e.g., the functionality to log in is only necessary because of authentication (a security requirement),
  - e.g., the system's ability to revert the system to a previous/back-up state is a consequence of an availability requirement, ..
  - e.g. the functionality to persist logs to a file is a consequence of a *traceability/accountability* security requirement
- ... that **in turn** can **introduce additional non-functional requirements**:
  - e.g. usability of the log in functionality
  - e.g. performance of the log in functionality

> Later topic on "**Twin Peaks model to SE**" zooms in deeper on this interplay between functional and non-functional requirements (and architecture)

Strategy 2. Identify quality requirements at the basis of

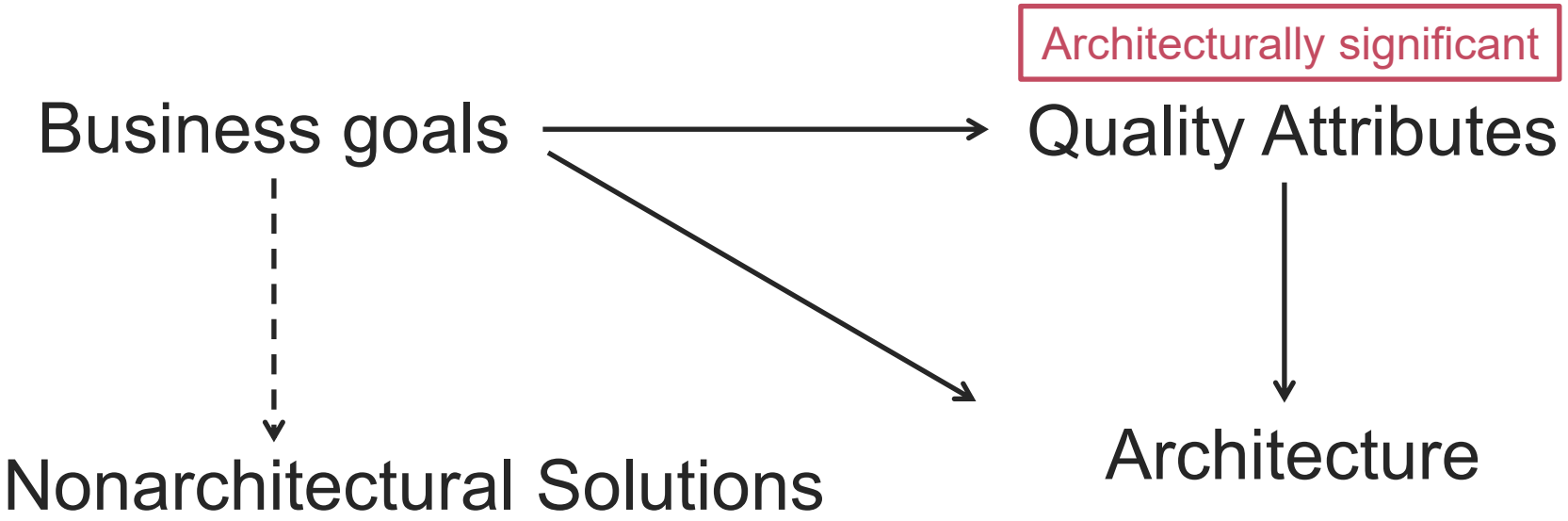
..the **business goals** of the system

Business goals



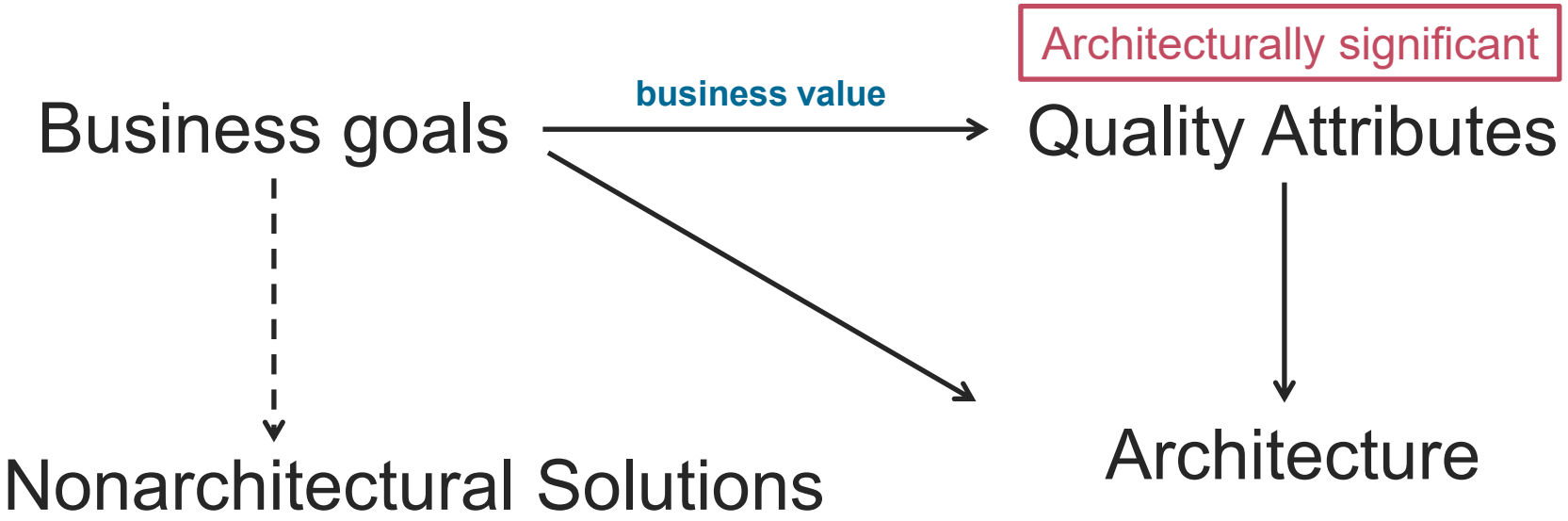
Software Architecture in Practice, ed 3; Chapter 3, Figure 3.2

Business goals



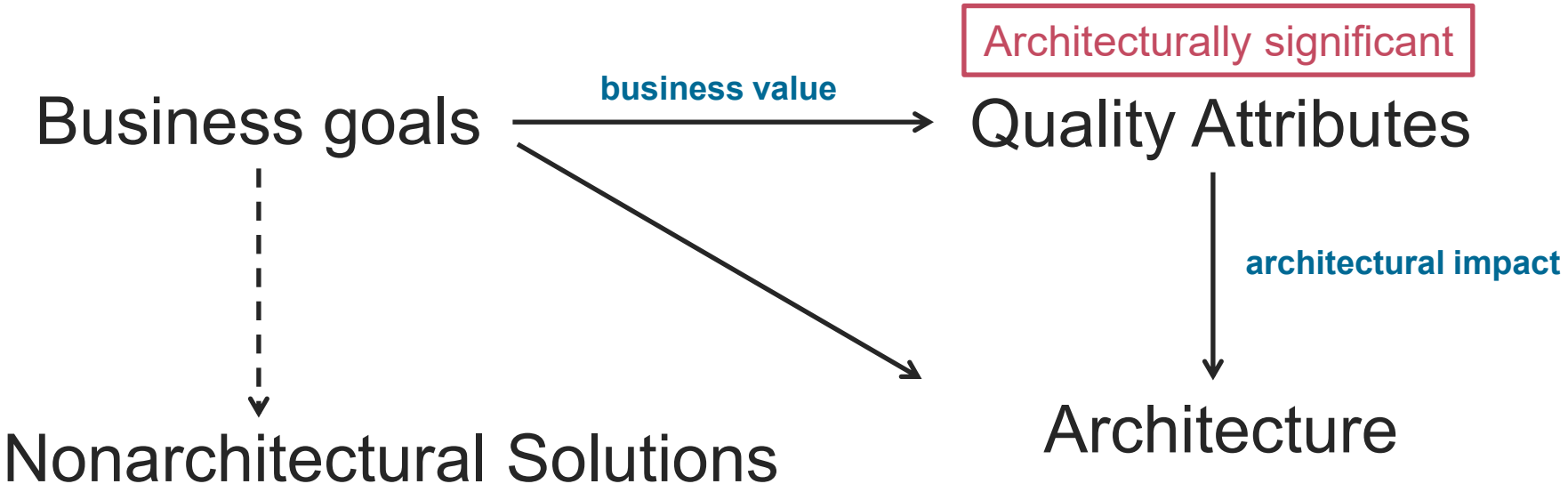
Software Architecture in Practice, ed 3; Chapter 3, Figure 3.2

Business goals



Software Architecture in Practice, ed 3; Chapter 3, Figure 3.2

Business goals



Software Architecture in Practice, ed 3; Chapter 3, Figure 3.2

Business goals

Gathering ASRs by understanding the business goals

› **Categories**

1. Contributing to the growth and continuity of the organization
2. Meeting financial objectives
3. Meeting personal objectives
4. Meeting responsibility to employees
5. Meeting responsibility to society
6. Meeting responsibility to state
7. Meeting responsibility to shareholders
8. Managing market position
9. Improving business processes
10. Managing the quality and reputation of products
11. Managing change in environmental factors

Software Architecture in Practice, ed 3; Chapter 16, Table 16.2

› ***Template*** (PALM):

1. Goal-source
2. Goal-subject
3. Goal-object
4. Environment
5. Goal
6. Goal-measure
7. Pedigree and value

Software Architecture in Practice, ed 3; Chapter 16

In this course:
good understanding of business goals is necessary, but they are documented informally

Business goals

Gathering ASRs by understanding the business goals

› **Cat**

1. Co
2. Me
3. Me
4. Me
5. Me
6. Meeting responsibility to state
7. Meeting responsibility to shareholders
8. Managing market position
9. Improving business processes
10. Managing the quality and reputation of products
11. Managing change in environmental factors

Strategy: you can first list the main business goals of the system, and reason from there about quality requirements.

Guiding questions:

- How does software quality contribute to meeting these goals?

- How may lack of software quality harm these goals?

6. Goal-measure
7. Pedigree and value

Software Architecture in Practice, ed 3; Chapter 16

In this course:
good understanding of business goals is necessary, but they are documented informally

Software Architecture in Practice, ed 3; Chapter 16, Table 16.2

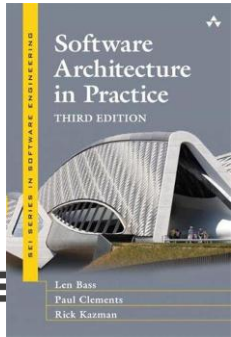
Strategy 3. Identify relevant quality requirements at the basis of

..types of **quality attributes**

Quality attributes

- Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

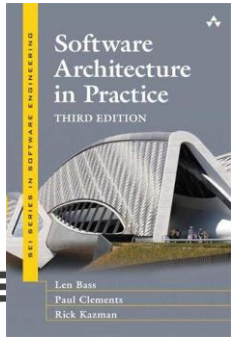


Quality attributes

- Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system

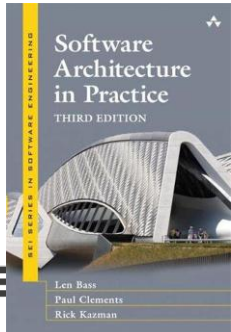


Quality attributes

- Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Performance (chapter 8)

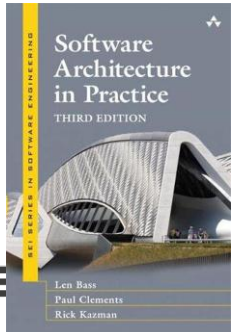
- "*It's about time*"
- "*Characterizing the event that can occur (and when they can occur) and the system or element's time-based response to those events is the essence of discussing performance*"

Quality attributes

- Primary: *Performance*, *Availability*, *Modifiability*, *Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Availability (chapter 5)

- "*Concept of reliability + the notion of recovery*"
- "*Refers to the ability of a system to mask or repair faults such that the cumulative service outage period does not exceed a required value over a specified time interval*"

- Fault = a system deviates from its specification ((in)*correctness*),
- Failure is when a system doesn't provide its service to the user or provides an unacceptably degraded service ~ accumulation of Faults and inability to mitigate leads to a situation of **effective loss**

Availability (chapter 5)

Robustness
Resilience

- "*Concept of reliability + the notion of recovery*"
- "*Refers to the ability of a system to mask or repair faults such that the cumulative service outage period does not exceed a required value over a specified time interval*"

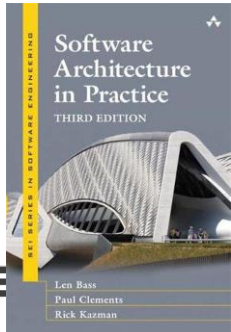
- Fault = a system deviates from its specification ((in)*correctness*),
- Failure is when a system doesn't provide its service to the user or provides an unacceptably degraded service ~ accumulation of Faults and inability to mitigate leads to a situation of **effective loss**

Quality attributes

- Primary: *Performance, Availability, Modifiability, Interoperability, Security, Testability, Usability*
- Secondary: *Variability, Portability, Development distributability, Scalability/elasticity, Deployability, Mobility,* and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Modifiability (chapter 7)

- "*Change happens*"
- "*Centers around the cost and risk of making changes*"
- Plan for future change

CAREFUL: **not** about our system being able to accept/process changes (~functionality),

nor is it about making changes now,

but about anticipating realistic future changes ~ evolution of our system over time

and addressing/limiting the potential impact of these changes to our system

Modifiability (chapter 7)

Extensibility

More scoped modifications
More incremental
Implies stronger preparations

- "*Change happens*"
- "*Centers around the cost and risk of making changes*"
- Plan for future change

CAREFUL: **not** about our system being able to accept/process changes (~functionality),

nor is it about making changes **now**,

but about **anticipating** possible future software modifications

~ evolution of our system over time

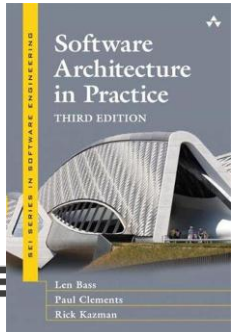
and proactively addressing/limiting the potential impact of these changes to our system

Quality attributes

- Primary: *Performance, Availability, Modifiability, Interoperability, Security, Testability, Usability*
- Secondary: *Variability, Portability, Development distributability, Scalability/elasticity, Deployability, Mobility,* and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Interoperability (chapter 6)

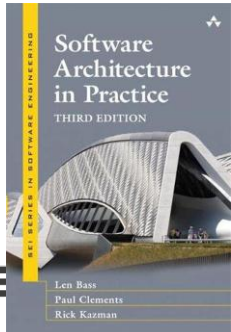
- "*A system cannot be interoperable in isolation*"
- "*Interoperability is about the degree to which two or more systems can usefully exchange meaningful information*"
  - "exchange information" ~ *interact, communicate, coordinate, participate, ..*

Quality attributes

- Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Security (chapter 9)

- "*A measure of the system's ability to protect data and information from unauthorized access while still providing access to people and systems that are authorized*"

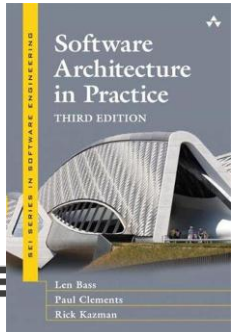
- CIA: Confidentiality, Integrity, Availability
- Threat modeling: mitigating threats:
  - STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

Quality attributes

- Primary: *Performance, Availability, Modifiability, Interoperability, Security, Testability, Usability*
- Secondary: *Variability, Portability, Development distributability, Scalability/elasticity, Deployability, Mobility,* and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Testability (chapter 10)

- "*A system is testable if it 'gives up' its faults easily*"

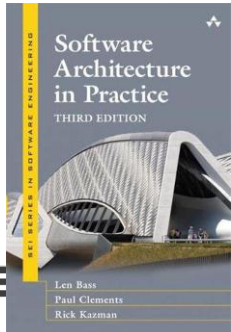
- Factors for functional testing:
  - Ability to control the inputs, and observe the outputs;
  - Ability to observe and control side-effects and private state,
  - Determinism of the system responses or outputs

Quality attributes

- Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Usability (chapter 11)

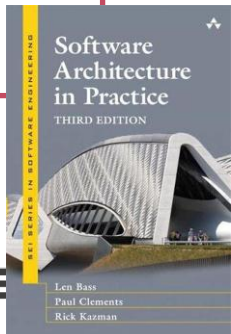
- "*User's perspective of quality*"
- "*Concerned with how easy it is for the user to accomplish a desired task*"
- End-user: human actor that directly interacts with our system
  - ~ similar to *Interoperability* but the actors are human and not external systems

Quality attributes

- Primary: *Performance, Availability*, *Modifiability, Interoperability*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

– see handbook

Strategy: you can go over the different types of quality attributes and and assess whether or not they will be relevant to the system



Other Quality Attributes

Chapter 12, SAiP

- **Variability**: the ability to support the production of *variants*
- **Portability**: the ease with which software that was built for one platform can be changed to run on a different platform
- **Development distributability**: Designing software to support distributed software development (cf. Modularity, Coupling, Cohesion, later)
- **Scalability**: the ability to effectively increase/decrease resources (horizontal/vertical), ratio #resources/#work

Other Quality Attributes

Chapter 12, SAiP

- **Deployability**: how an executable arrives at a host platform and how it is invoked; the ability to do this in acceptable time and effort
- **Mobility**: problems and affordances of a platform (e.g. battery management/resource consumption, platform requirements, display size, etc)
- **Monitorability**: the ability of the operations staff to monitor the system while it is executing (health, queue lengths, average transaction processing time, etc)
- **Safety**: the ability to avoid entering states that may cause or lead to damage, injury, loss of life to actors in the environment (and limit the damage when it does enter into bad states)

Other Quality Attributes

Edition 4, SAiP

- **Integrability**: the costs and technical risks of anticipated and (to varying degrees) unanticipated future integration tasks.
- **Energy-efficiency**: the ability to reduce energy consumption or keep it within bounds

Recap: three complementary strategies

Identify relevant quality requirements at the basis of

1. …*the functionality of the system*
2. …*business goals*
3. …types of *quality attributes*

+ thinking from the perspectives of the different stakeholders (developer, customer, end-user, government, …)
+ …

Part II.

Once you have identified the relevant quality attributes for the system under analysis

how to document or communicate them?

Consider the following non-functional requirements

- “*The system should be very secure*”
- “*We want high availability*”
- “*The database should scale well*”
- “*DocProc Dashboard should be easy-to-use*”
- “*DocProc should be interoperable with Zoomit*”

“*The system should be very secure*”

- Which part will be most vulnerable?
- Confidentiality? Integrity? Availability?
- Against which adversary? *Script kiddie* that tries out a few attacks versus a powerful nation state?
- Which assets to protect? What is their value?

When do we meet the requirement?

We want high availability

- Large difference in architectural solution between “99% availability and 99.999% availability”

Availability	Downtime/90 days	Downtime/year
99,0%	21 hrs, 36 mins	3 days, 15,6 hrs
99,9%	2 hrs, 10 mins	8 hrs, 0 mins, 46 sec
99,99%	12 min, 58 sec	52 min, 34 sec
99,999%	1 min, 18 sec	5 mins, 15 sec
99,9999%	8 seconds	32 seconds

When do we meet the requirement?

“Our database should scale well”

- Scale in #data records? (Storage size)
  - large volumes of data per patient?
  - many patients?
- Scale in terms of #data synchronizations/ time unit? (Write operations)
- Scale in terms of computation? (Read/write)
  - # document generation jobs?
  - # batches, running parallel or in sequence?

Scalability != performance

When do we meet the requirement?

“*Dashboard should be easy-to-use*”

- To do what?
  - To upload a new batch?
  - Fix a problem in raw data?
  - Tweak a template?
  - Follow up on document delivery?
  - ..?

Our Document Processing system should be interoperable with Zoomit (context: another example)

- Deliver documents one-by-one or in batch? Possible?
- APIs are a given, we cannot change them (**system context!**)

- Normal versus exceptional cases
- How are errors dealt with?
- What about edge cases? (e.g. Zoomit has accepted a document but user unregisters, Zoomit id was valid at time of upload, but user has deleted account in meantime)

When do we meet the requirement?

The real challenge is not in finding relevant requirements...
it is in being **precise** and **concrete**
and to ensure **verifiability** of your quality requirements

Desired characteristics of a NFR spec

- Objective (unambiguous and clear) & specific (precise)
  - *What does it apply to? All aspects of the system or some delineated subsystems?*
  - *Are we talking about a process crashing on a server or an entire data center out of power?*
- Objectively measurable or verifiable
  - Concrete **metrics** (if possible, depends on the nature of the quality attribute)
  - **Booleans** and **conditions** (if metrics aren't available)

Can you articulate an objective criterion that allows someone else to verify whether or not the requirement is met?

- ASRs and QASs will both help you in this regard because of their structure/form/template

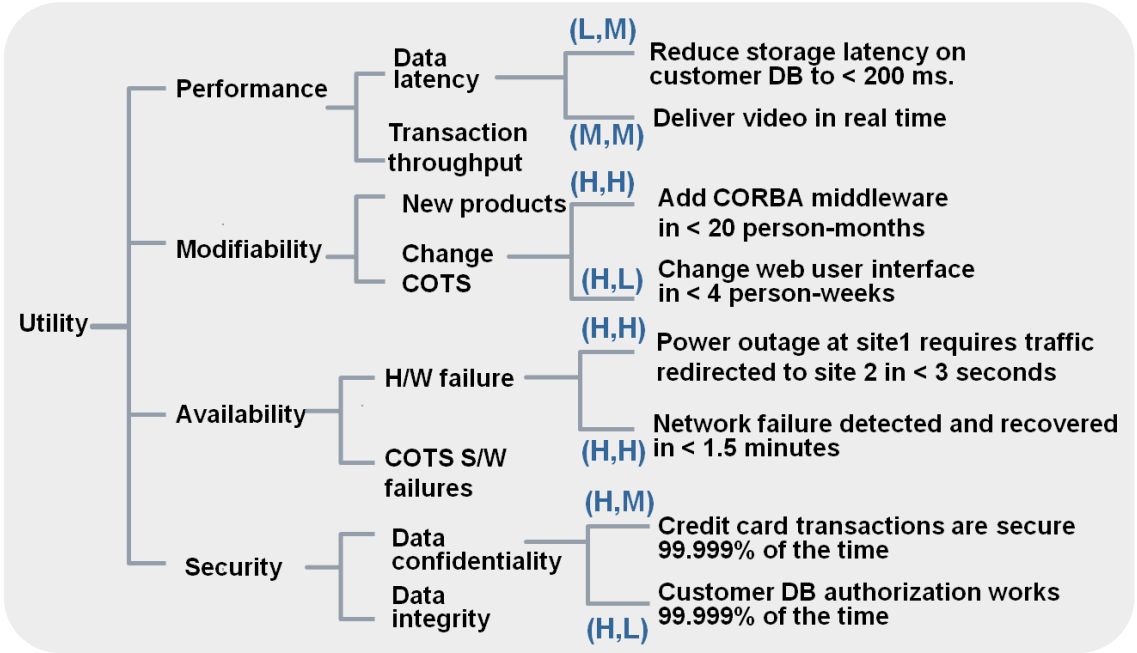
Relevant metrics

Quality attribute	Metrics to express quality response
Availability	uptime, mean time between failure, etc.
Performance	throughput, latency
Modifiability	time (man hours), economic cost
Usability	tasks/seconds, human errors made
Interoperability	percentage of successful/unsuccessful information exchanges
Security	time to detect attacks, how much of the system was compromised, amount of attacks resisted, amount of data shown vulnerable during attack
Testability	effort to find a fault, to achieve a percentage of state space coverage, probability of faults being revealed by the next test, test time, length of the longest dependency chain in test, time to prepare test environment, reduction in risk exposure.

ASRs and Utility tree

Utility tree

- Four levels:
  1. Utility: expression of the overall *goodness* of a system
  2. Quality attribute
  3. Quality attribute refinement
  4. ASR



Architecturally significant requirements (ASRs)

ASR description

- Free-form text but **includes an objective criterion** (metric or condition); [1 to a few sentences]
- Easier to have **fine-grained ASRs** to allow distinguish between different aspects
- E.g. "*system availability should be > 99.999%*" versus "*database availability should be >99,999%*" and "*front-end availability should be > 99%*"

Architecturally significant requirements (ASRs)

ASR description

- Free-form text but **includes an objective criterion** (metric or condition); [1 to a few sentences]
- Easier to have **fine-grained ASRs** to allow distinguish between different aspects
- E.g. "*system availability should be > 99.999%*" versus "*database availability should be >99,999%*" and "*front-end availability should be > 99%*"

Especially for High-Value and High-Impact requirements, further refinement is a good idea

Architecturally significant requirements (ASRs)

- Criteria:
  1. **Business or mission value:** importance to stakeholders
  2. **Architectural impact**: including the ASR will result in a very different architecture than if it were not included
- Using a single list can help to evaluate each potential ASR against these criteria **and prioritize**

Utility tree

- Quality attribute
- Refinement
- Description
- Business value (H,M,L)
- Architectural impact (H,M,L)

Utility tree

- **Quality attribute**
- **Refinement**
- **Description**
- **Business value (H,M,L)**

1/ **Correct** classification?

2/ **Diversity** (Availability, Modifiability, Performance, Security, Testability, Usability, Interoperability, +more)

Utility tree

- **Quality attribute**
- **Refinement**
- **Description**
- **Business value (H,M,L)**

1/ **Clear**?
2/ **Focused** on the most **important aspects**?
3/ **Expected/creative**?

Utility tree

- **Quality attribute**
- **Refinement**
- **Description**
- **Business value (H,M,L)**

1/ **Clear, concrete** and **precise**?
2/ **Correct labeling**?
3/ **Measurable or verifiable** or **hint towards how to assess it**?

Utility tree

- **Quality attribute**
- **Refinement**
- **Description**

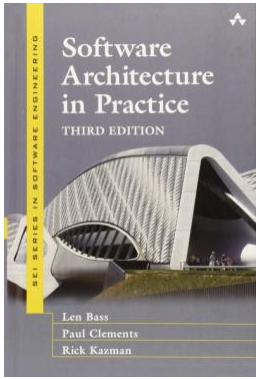
1/ **Sound reasoning** in terms of the target?
2/ **Understandable rationale**?
3/ **Acceptable ranking**?

- **Business value (H,M,L)**
- **Architectural impact (H,M,L)**

Quality Attribute Scenarios

Quality attribute scenario elicitation

- Many *-ilities*.
- Handbook covers these
  - Primary: *Availability*, *Interoperability*, *Modifiability*, *Performance*, *Security*, *Testability*, *Usability*
  - Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*



Quality Attribute Scenarios: 6 parts

- **Source** of stimulus (human, computer systems, any actor)
- **Stimulus** (condition that must be considered)
- **Artifact** (which part of the system is stimulated?)
- **Environment** (accompanying conditions, context)
- **Response** (activity undertaken, i.e., what expected by a stakeholder)
- Response **measure** (enabling acceptance of response/satisfaction of requirement)

Examples

Performance Modifiability

1. Source: Developer

2. Stimulus: Wishes to Change the UI

3. Artifact: Code

4. Environment: at development time

5. Response: Modification is made with no side effects

6. Response Measure: in 3 days

1. Source: User

2. Stimulus: Initiate transaction

3. Artifact: System

4. Environment: Under normal operation

5. Response: Transaction is processed

6. Response Measure: With average latency of 2 sec

Scenario generation

- Handbook provides checklists to guide the systematic elicitation

- Tables with templates to be used to derive system-specific scenarios
  - Per quality attribute
  - What sources to consider? What measures?

Responses and Response measures

- **Prescribe** how should the system behave given the stimulus
- **Dictates** measurable and verifiable constraints

- **Tables** are good for initial brainstorm/determining the relevant aspects (e.g. Availability: prevent, detect, recover)
  - But these are not exhaustive
  - Be creative, rely on expertise and knowledge (from books, other courses, …)
    - Performance, Availability: Distributed Systems
    - Modifiability, Extensibility, etc: OASS/SWOP/…
    - Security: …

Performance	Possible Values
Source	Internal or external to the system
Stimulus	Arrival of a periodic, sporadic, or stochastic event
Artifact	System or one or more components in the system.
Environment	Operational mode: normal, emergency, peak load, overload.
Response	Process events, change level of service
Response Measure	Latency, deadline, throughput, jitter, miss rate

Table 8.1, SAiP[82]

Availability	Possible Values	
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment	
Stimulus	Fault: omission, crash, incorrect timing, incorrect response	
Artifact	System's processors, communication channels, persistent storage, processes	
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation	
Response	**Prevent** the fault from becoming a failure **Detect** the fault: • log the fault • notify appropriate entities (people or systems)	**Recover** from the fault • disable source of events causing the fault • be temporarily unavailable while repair is being effected • fix or mask the fault/failure or contain the damage it causes • operate in a degraded mode while repair is being effected
Response Measure	• Time or time interval when the system must be available • Availability percentage (e.g. 99.999%) • Time to detect the fault • Time to repair the fault • Time or time interval in which system can be in degraded mode • Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing	

Modifiability	Possible Values
Source	End user, developer, system administrator
Stimulus	A directive to add/delete/modify functionality, or change a quality capacity, or technology
Artifact	Code, data, interfaces, components, resources, configurations, …
Environment	Runtime, compile time, build time, initiation time, design time
Response	One or more of the following: • make modification • test modification • deploy modification
Response Measure	Cost in terms of: • number, size, complexity of affected artifacts • effort • calendar time • money (direct outlay or opportunity cost) • extent to which this modification affects other functions or quality attributes • new defects introduced

Table 7.1, SAiP

Interoperability	Possible Values
Source	A system initiates a request to interoperate with another system
Stimulus	A request to exchange information among system(s)
Environment	The systems that wish to interoperate
Artifacts	System(s) wishing to interoperate are discovered at runtime or known prior to runtime.
Response	One or more of the following: • The request is (appropriately) rejected and appropriate entities (people or systems) are notified. • The request is (appropriately) accepted and information is exchanged successfully. • The request is logged by one or more of the involved systems.
Response Measure	One or more of the following: • percentage of information exchanges processed successfully, • percentage of information exchanges correctly rejected

Table 6.2, SAiP

Usability	Possible Values
Source	End user, possibly in a specialized role
Stimulus	End user tries to use a system efficiently, learn to use the system, minimize the impact of errors, adapt the system, or configure the system
Artifact	System or the specific portion of the system with which the user is interacting.
Environment	Runtime or configuration time
Response	The system should either provide the user with the features needed or anticipate the user's needs.
Response Measure	One or more of the following: task time, number of errors, number of tasks accomplished, user satisfaction, gain of user knowledge, ratio of successful operations to total operations, or amount of time or data lost when an error occurs.

Table 11.1, SAiP

Security

Table 9.1, SAiP

Security	Possible Values
Source	Human or another system which may have been previously identified (either correctly or incorrectly) or may be currently unknown. A human attacker may be from outside the organization or inside the organization
Stimulus	Unauthorized attempt is made to display data, change or delete data, access system services, change the system's behavior, or reduce availability.
Artifact	System services, data within the system, a component or resource of the system, data produced or consumed by the system
Environment	The system is either offline or online; either connected to or disconnected from a network; either behind a firewall or open to a network; fully operational, partially operational, or not operational.
Response	Transactions are carried out in a fashion such that: • Data or services are protected from unauthorized access. • Data or services are not being manipulated without authorization. • Parties to a transaction are identified with assurance. • The parties to the transaction cannot repudiate their involvements. • The data, resources, and system services will be available for legitimate use. The system tracks activities within it by: • Recording access or modification • Recording attempts to access data, resources, or services • Notifying appropriate entities (people or systems) when apparent attack is occurring
Response Measure	One or more of the following: • How much of a system is compromised when a particular component or data is compromised • How much time passed before an attack was detected • How many attacks were resisted • How long does it take to recover from a successful attack • How much data is vulnerable to a particular attack

Testability	Possible Values
Source	Unit testers, integration testers, system testers, acceptance testers, end users, either running tests manually or using automated testing tools
Stimulus	A set of tests is executed due to the completion of a coding increment such as a class, layer or service, the completed integration of a subsystem, the complete implementation of the whole system, or the delivery of the system to the customer.
Environment	Design time, development time, compile time, integration time, run time
Artifact	The portion of the system being tested.
Response	One or more of the following: • Execute test suite and capture results • Capture activity that resulted in the fault, • Control and monitor the state of the system
Response Measure	One or more of the following: • Effort to find a fault or class of faults, • Effort to achieve a given percentage of state space coverage, • Probability of fault being revealed by the next time, • Time to perform tests, • Effort to detect faults, • Length of longest dependency chain in test, • Length of time to prepare test environment, • Reduction in risk exposure (size(loss)*prob(loss))

Table 10.1, SAiP

What do you think?

about the following ASRs

27	Interoperability	Reliable Interoperation	We maintain a high reputation so that our mails are not sent to spam.
			H: *If customers do not receive their invoices we may have violated the SLA.*
			L: *This is mostly a sysadmin matter and we can use a high-reputation email provider.*
28		Reliable Interoperation	It takes at most 2 man-days to adapt to changes in a delivery service's interface.
			H: *Outages in delivery of invoices are to be avoided at all costs.*
			M: *Sufficient decoupling is necessary to support even significant changes in external interfaces.*

24	Performance	Documents processed in time	99.9 % of all batches have to be processed within the time specified in the SLA.
			H: *We have to keep our promise or we will get a bad reputation.*
			H: *We have to think about this performance throughout the architecture.*
25		Registration	When a customer or Recipient registers, he/she should be able to access their portal within 10 seconds.
			M: *Users who register usually do this to use the system immediately afterwards.*
			M: *The back-end might need some "triggers" to make this possible.*

What do you think?

about the following quality attribute scenarios

What do you think?

M3: Register a new patient
A new patient is registered
Source: Nurse
Stimulus: New patients with a heart condition needs to be monitored
Artifact: This modification concerns nurses, they have the task of adding patients
Environment: At run-time
Response:
- A patient registers the new patient
- The patients gets the necessary information, a questionnaire to fill in and gets the right devices to monitor him

Response measure:
- Adding a patient takes no longer than 5 minutes

What do you think?

Implementing the system in a different hospital

Source	New hospital information system
Stimulus	Offering the same functionality with a different hospital information system
Artifact	The communication between the patient monitoring system and the hospital information system
Environment	Building the system in a new hopsital
Response	• Meetings should be organized with the people responsible for the local HIS. • The system should not have to change because of the different interface to the HIS
Response measure(s)	• Industry standards should be employed in communication with the HIS. • Communication should happen completely architecture independent • A well defined interface of what the PMS requires of the HIS should be presented to possible future clients. • Deployment costs and time should be kept as low as possible.

What do you think?

5.2.3 Modify Functionality

Source:	Developer
Stimulus:	Is contacted by the hospital to modify or extend some functionality (for instance: adding a new risk level)
Artifact:	Patient Monitoring System
Environment:	At runtime
Response:	1. Locates places in architecture to be modified. 2. Makes modification without affecting other functionality. 3. Tests modification. 4. Schedules downtime. 5. Deploys modification.
Response measure:	• Maximum 2 hours of downtime required for deploying the modifications. • The gateways can still send new data packages to the system, these are stored to be processed later.

What do you think?

5.2 Performance

5.2.1 Idle load of the system

Source

Users

Stimulus

50 users are using the system simultaneously, resulting in low CPU usage.

Artifact

System's services

Environment

Normal operation

Response

The system should process every arriving request fast. The requests should be processed according to the arriving order of the requests. We consider idle load when the CPU usage is below 40%.

Response measure

The request should be processed in 10ms.

What do you think?

A physician is unavailable for a certain prolonged period

Source	External to the system
Stimulus	A physician responsible for a number of patients in the monitoring system is unavailable for a certain period, be it due to vacation or sickness.
Artifact	Physician
Environment	Normal Operation
Response	• When a physician plans a period of unavailability in the HIS, our system should notice this. • Our system should take note of the replacing physician and temporarily appoint him all responsibilities of the patients. • The original physician should still be notified of events. • The patient should be notified through the gateway that his physician is unavailable for any questions and should be given the information of his replacement
Response measure(s)	The HIS should supply an easy way of obtaining physician availability information.

What do you think?

Quality attribute scenario 3 Availability 3

Source Data connection of gateway

Stimulus The ISP is having some technical difficulties, or in case of a 3G connection: bad coverage in this area.

Artifact Data connection

Environment Normal operation time

Response

- The system should detect a loss of connection, while clearly separating it from a crash of the gateway. (see scenario 1)
- The gateway must cache the data to send.
- When the connection becomes available again, the gateway attempts to send the data again.
- If success, remove cached data.

Response Measure

- Packet loss shall dealt with at all times. A packet will be resent when connection is available.

Positive examples

Some initial requirements for the project already provided in the assignment 1 template:
Performance + Modifiability ASRs
and one corresponding Modifiability QAS

What is software architecture?

This week's definition of **software architecture**

- Centered around the notion of utility
  - As expressed in the utility tree,
  - A weighted combination of the different quality requirements
- Software architecture = meeting the *utility* requirements
  - Architecture design process = coming up with different candidate designs,
  - each with their own utility score, and identifying the best candidate (~ search/optimization problem; utility as a form of *fitness*)

Software architecture, definitions so far

- Lecture 1: "*the bridge between requirements and design*"
  - (quite informal)
- Lecture 2: "*meeting the overall utility expressed as a combination of quality attribute requirements*"

Thank you
… and let’s start Part 1.